



Scientific Software Ecosystem for HPC

From Physical Models to Large-Scale Simulations

Mushafia Sadia Aman

Researcher, HPC Systems for Modeling and Simulations

Centre for AI and Big Data, Namal University Mianwali

Contents

1

Why scientific software exists

2

The layered software stack

3

Molecular dynamics — GROMACS

4

Quantum materials — Quantum ESPRESSO

5

Fluid dynamics — OpenFOAM

6

Running applications on the cluster

7

Monitoring & performance on HPC

8

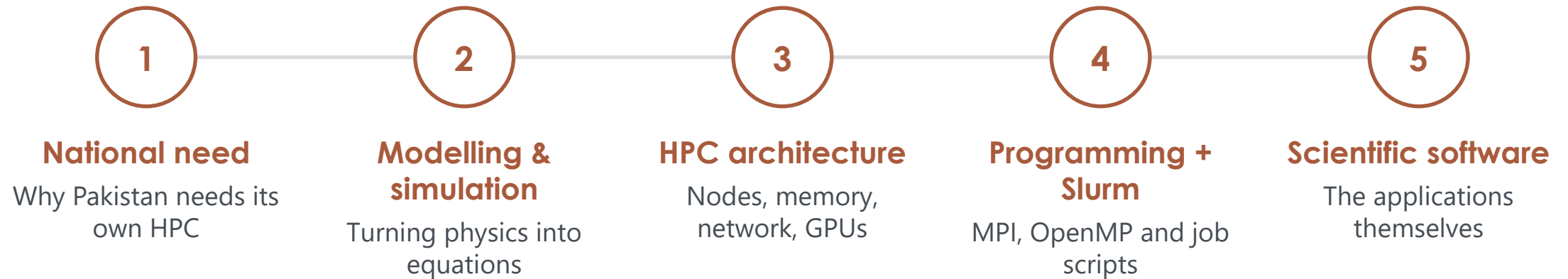
Takeaways & hands-on preview

What You Will Take Away

- How scientific software turns a physical model into a running simulation
- Which package fits which physics — GROMACS, Quantum ESPRESSO, OpenFOAM, The end-to-end workflow of molecular dynamics, DFT and CFD runs
- How MPI, OpenMP, GPUs, numerical libraries and Slurm sit underneath them
- How to monitor a running job and read its performance before scaling up
- Why reproducibility and benchmarking come before production runs

Pick the right tool → run it correctly → prove it ran well

Where Are We in the HPC Journey?



WHAT TODAY ADDS

Which software solves which problem?

How does it actually use the cluster?

What runs in the hands-on session?

The Scientific Software Ecosystem

What these tools are, why they exist, and how the stack fits together



Why Scientific Software?

Scientists should focus on the science — not rebuild every solver from scratch.

Physics and models

Force fields, governing equations, boundary conditions and electronic-structure models — already validated by the community.

Numerical methods

Time integration, iterative and sparse solvers, FFTs, linear algebra, meshing and optimisation, implemented and tested.

HPC execution

Parallel algorithms, CPU vectorisation and GPU offload, checkpointing, scalable I/O and native MPI / OpenMP interfaces.

Scientific software = validated models + numerical algorithms + optimised implementation

The Scientific Software Stack

01

Scientific problem

Protein, material, airflow,
structure



02

Application

GROMACS · Quantum
ESPRESSO · OpenFOAM



03

Numerical algorithms

Integrators · linear algebra ·
FFT · PDE solvers



04

Libraries & runtime

BLAS/LAPACK · MPI ·
OpenMP · CUDA



05

Scheduler & environment

Slurm · modules · containers



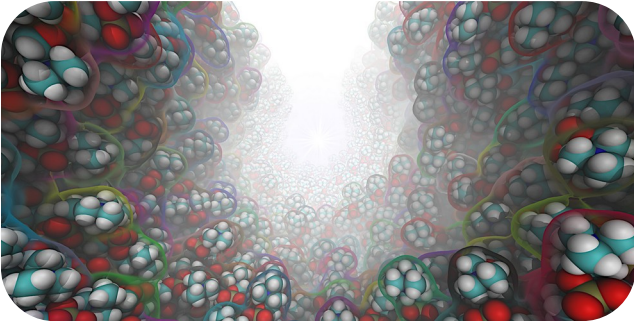
06

HPC hardware

CPU · GPU · memory ·
network · storage



Three Domains, Three Software Families

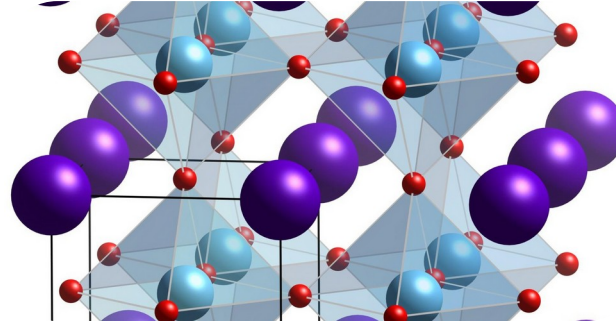


Molecular & atomistic

GROMACS

Proteins, biomolecules,
materials, atomic motion.

nm scale · femtosecond steps

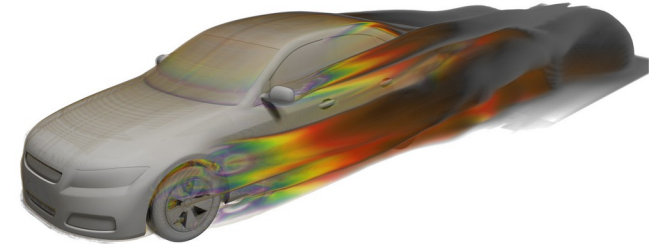


Quantum & materials

Quantum ESPRESSO

Electronic structure,
semiconductors, catalysts.

Å scale · electrons and bonds



Fluids & engineering

OpenFOAM

CFD, heat transfer,
turbomachinery, aerodynamics.

cm–m scale · continuum fields

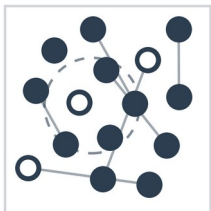


All packages shown are open source; licensed alternatives such as VASP and ANSYS exist in each domain.

From Modeling Approach to Software

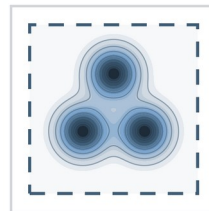
Modeling approach	Typical application	Representative software
Particle / molecular	Molecular dynamics	GROMACS, LAMMPS
Quantum / electronic	DFT, electronic structure	Quantum ESPRESSO
Mesh / continuum	CFD, heat transfer	OpenFOAM

HOW EACH APPROACH REPRESENTS THE SYSTEM



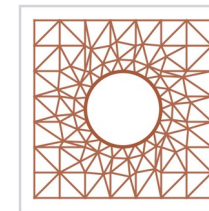
Particle / molecular

Atoms tracked individually;
forces from near neighbours.



Quantum / electronic

Electron density solved on a
grid inside a periodic cell.



Mesh / continuum

Fields averaged over cells that
fill the domain.

Choose by the physics and numerical method first — not by which package is most popular.

How to Choose Scientific Software

Scientific fit

- Physics and model assumptions
- Numerical method and discretisation
- Accuracy and published validation
- Problem scale — atoms, cells, k-points
- Whether the result can be reproduced

Practical fit

- Parallel scalability on cases
- GPU support in the build
- Compatibility with the cluster MPI and compiler
- Licensing and access to the source
- Community, documentation and support

Three questions that settle most choices

Is it valid?

Has this package been validated for physics?

Does it scale?

Does it hold efficiency at the size I actually run?

Can I rerun it?

Will this exact version and build still exist next year?



Molecular Dynamics — GROMACS

Following atoms through time: forces, trajectories and what they tell us



Molecular Dynamics

Track atoms by repeatedly computing forces and moving them forward in time.



Compute forces

Evaluate the force on every atom from the force field and its neighbours.

Integrate motion

Solve Newton's equations for one femtosecond-scale time step.

Update state

Write new positions and velocities; rebuild neighbour lists as needed.

Advance & repeat

Loop millions of times to build a trajectory of the system.

Scientific output: structure, energy, temperature, pressure, diffusion, binding and conformational change

What it is used for

- Proteins, enzymes and biomolecules
- Lipid membranes and drug–protein systems
- Solvated molecular systems
- Polymers and soft matter
- Free-energy and binding studies

Why it fits HPC

- Heavily vectorised CPU kernels
- OpenMP threading within a node
- MPI domain decomposition across nodes
- GPU offload of non-bonded forces
- Built-in trajectory analysis tools

Why molecular dynamics needs HPC

More atoms

More pair interactions and larger neighbour lists each step.

Smaller time steps

Millions of femtosecond steps for nanoseconds of real time.

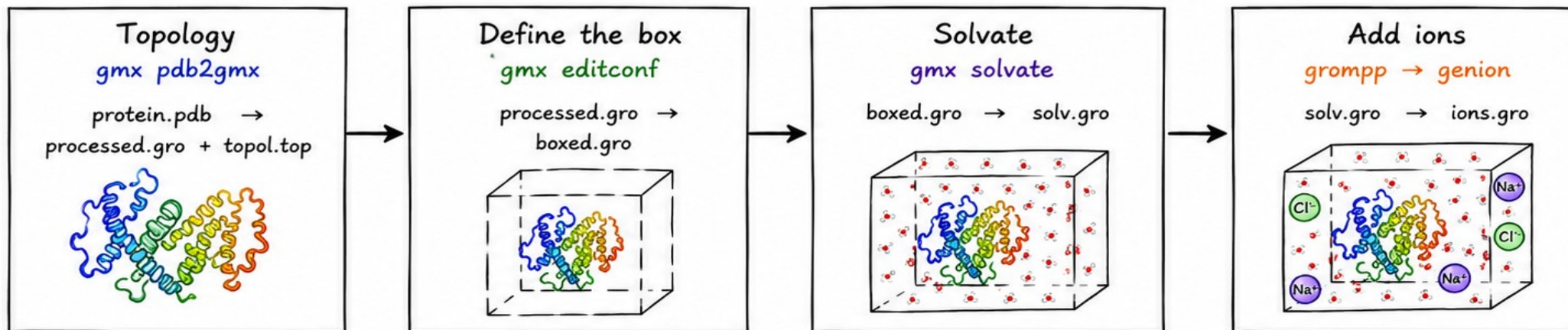
Many replicas

Ensembles and parameter sweeps multiply the work.

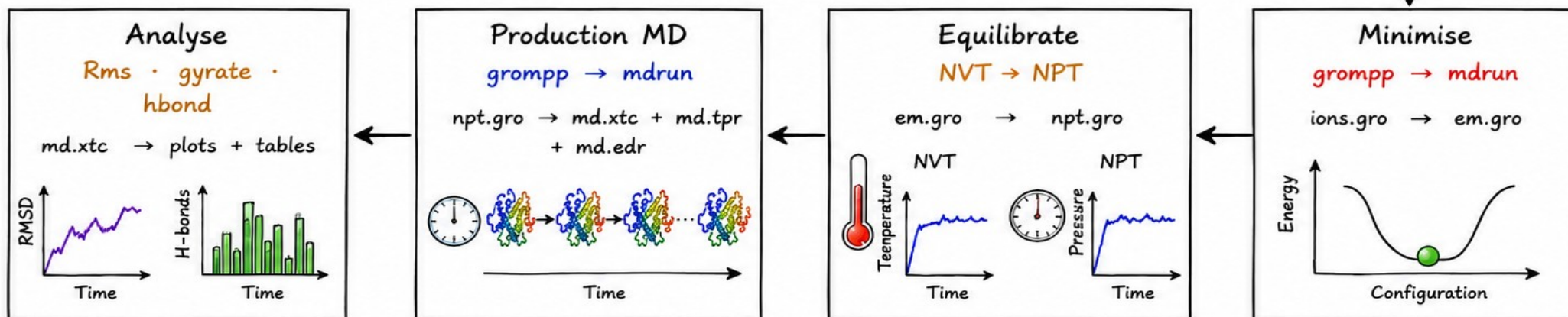
The GROMACS Workflow

A standard molecular-dynamics pipeline from structure preparation to trajectory analysis

SYSTEM PREPARATION



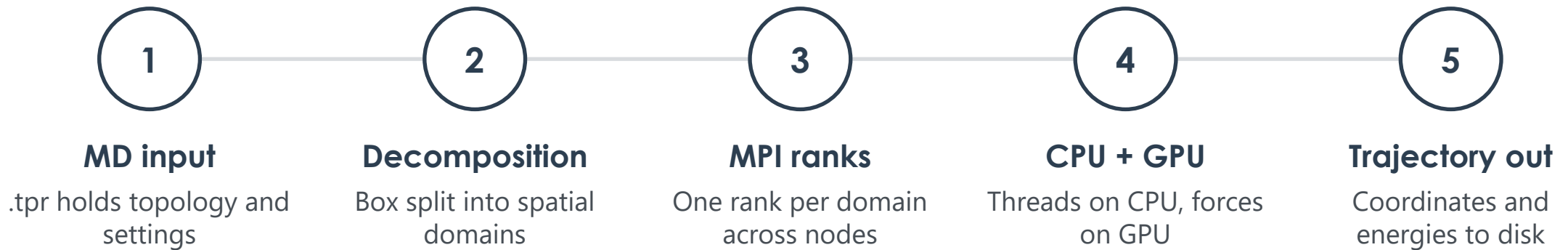
SIMULATION & ANALYSIS



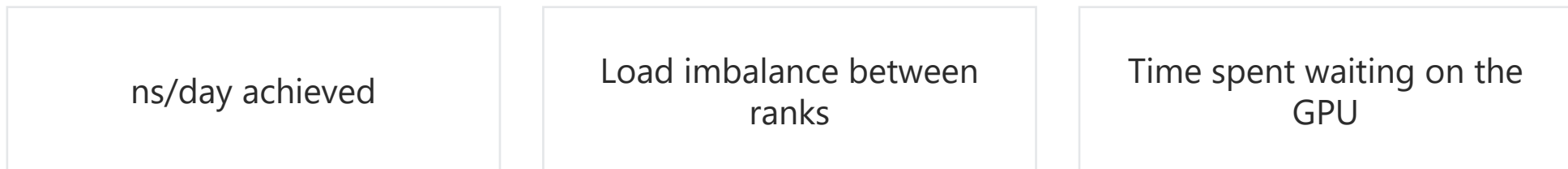
Simulation stages reuse the same execution pattern: `.mdp` + `.gro` + `.top` → `grompp` → `.tpr` → `mdrun`

Where Does HPC Enter GROMACS?

The same input file runs on a laptop or on 8 nodes — the difference is how the work is divided.



WHAT THE LOG FILE TELLS YOU



Performance depends on the balance between MPI ranks, CPU threads, GPU offload and communication.

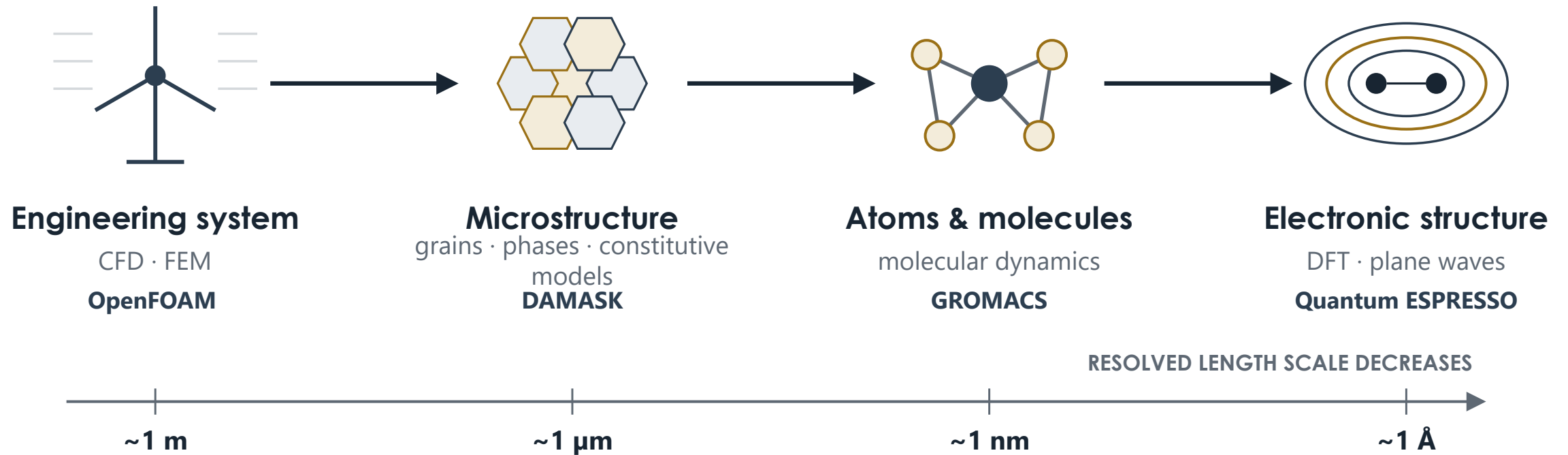


Quantum & Materials — Quantum ESPRESSO

Electronic structure from first principles: different physics, different parallelism

From Engineering to Electronic Scale

As the resolved length scale shrinks, the numerical model changes with it



Smaller length scale → more explicit degrees of freedom → greater computational cost

Quantum ESPRESSO

Typical calculations

- Self-consistent field (SCF) energies
- Structural relaxation
- Band structure and density of states
- Phonons and lattice dynamics
- Plane-wave pseudopotential methods

HPC characteristics

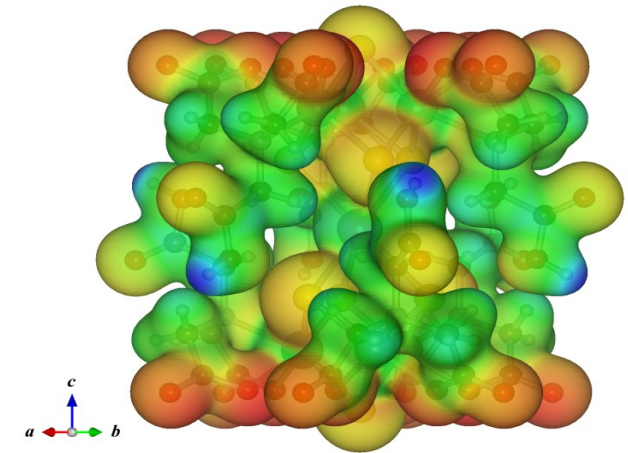
- MPI and OpenMP parallelism
- FFT-intensive workloads
- Dense and distributed linear algebra
- Parallel over k-points and bands
- High memory and communication demand

Why DFT becomes computationally expensive

More k-points — Denser Brillouin-zone sampling multiplies the calculations.

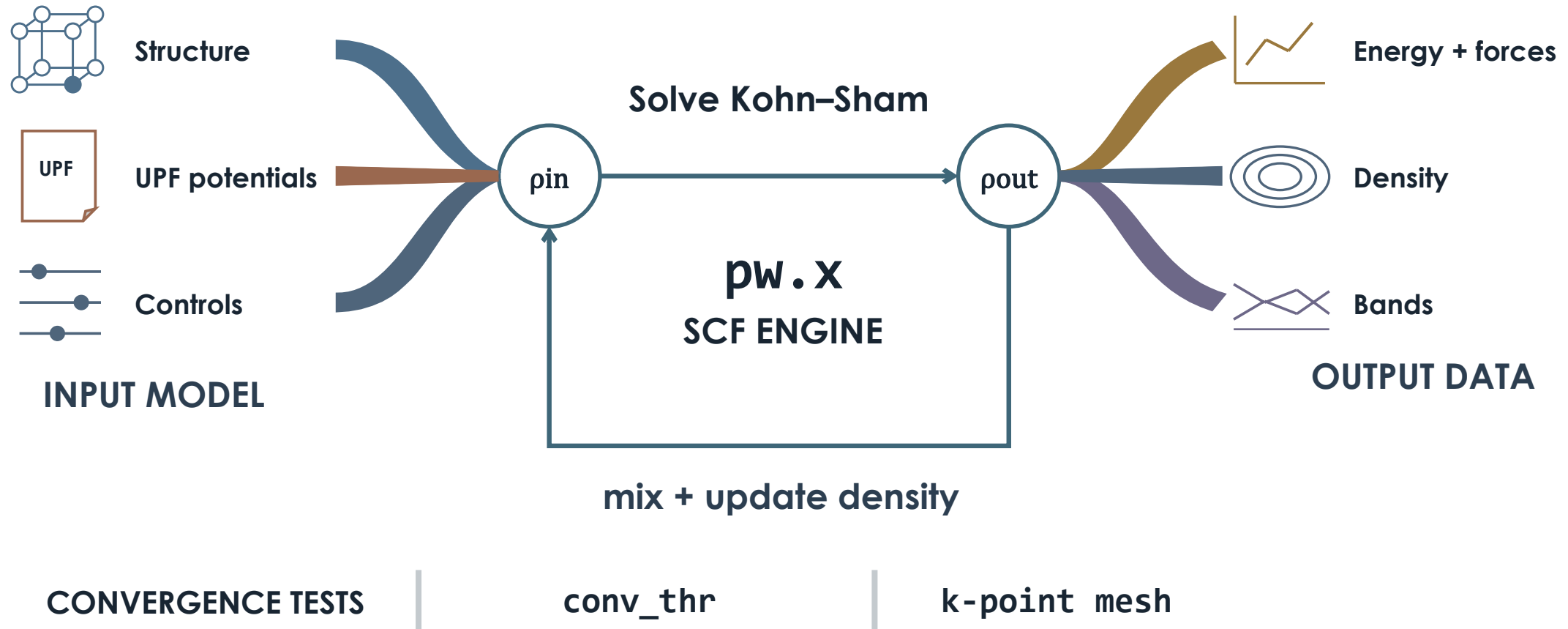
More electrons — Larger cells mean more states and bigger matrices.

Higher cutoff — More plane waves raise FFT and linear-algebra cost.



Quantum ESPRESSO: The SCF Feedback Loop

SCF is a fixed-point iteration on the electron density



Parallelism in Quantum ESPRESSO

k-point parallelism

Different k-points are largely independent — the easiest and most efficient level to parallelise.



Band / state parallelism

Electronic states are distributed across groups of processes within a pool.



FFT parallelism

Plane-wave grids need distributed FFTs — heavy all-to-all communication.

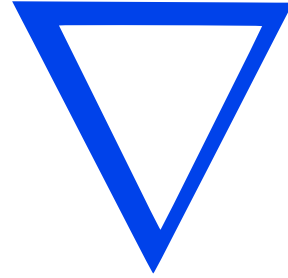


Linear algebra

Diagonalisation and matrix operations lean on optimised ScaLAPACK / ELPA libraries.



The right mix of pools, bands and FFT groups is what makes a DFT job scale.



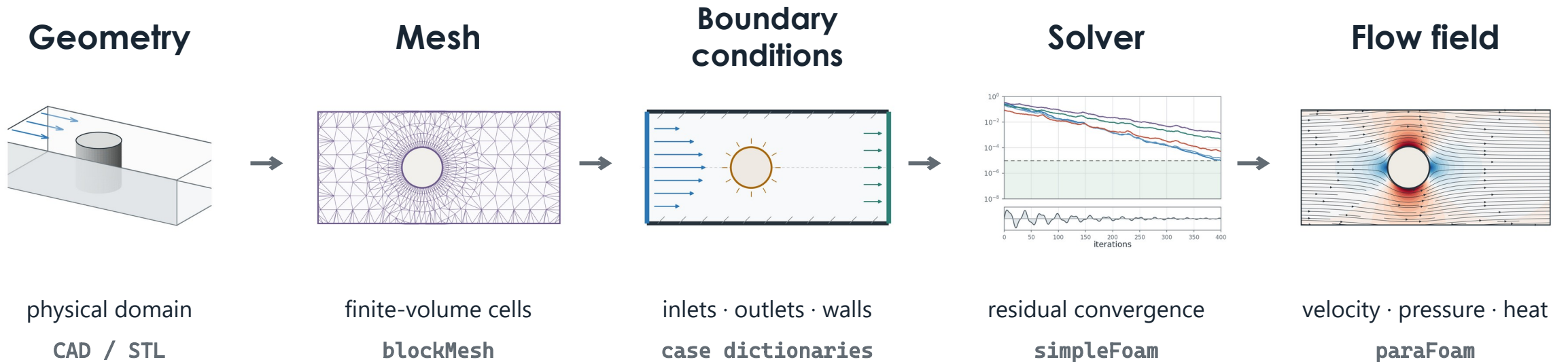
Fluid Dynamics — OpenFOAM

Mesh-based simulation of flow, heat transfer and engineering systems



CFD: From Geometry to Flow Field

The same domain is discretized, constrained, solved, and visualized



WHAT THE FLOW FIELD ANSWERS

How does the fluid move?

Where is pressure highest?

How much heat is transferred?

Mesh quality and solver settings determine accuracy, stability, and computational cost

Where it is applied

- Aerodynamics and external flow
- Fans, pumps and turbomachinery
- Heat transfer and cooling
- Water and environmental flow
- Combustion and process industry

Why it needs HPC

- Meshes with millions of cells
- Several coupled equations per cell
- Iterative sparse linear solvers
- Transient runs with many time steps
- MPI domain decomposition

What makes a CFD run expensive

Mesh size

Accuracy demands finer cells — and cell count drives everything.

Time steps

Transient problems need thousands of small, stable steps.

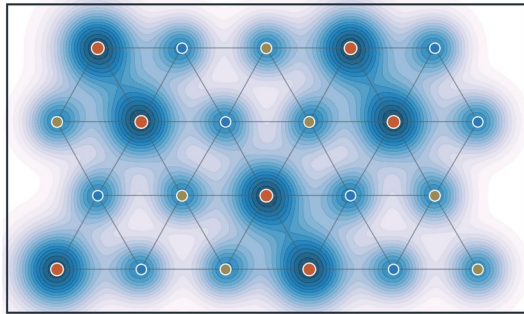
Communication

Neighbouring subdomains swap boundary values every step.

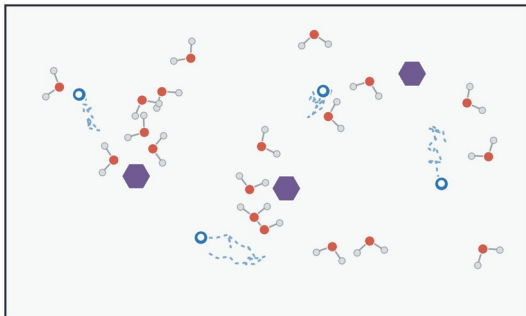
One Problem, Four Tools

A battery pack spans physics from ångströms to metres

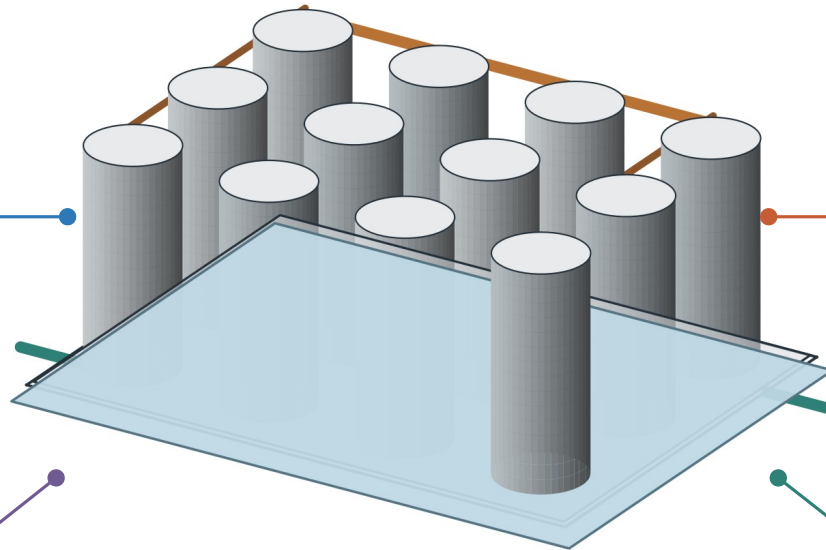
Electronic structure
Quantum ESPRESSO · Å
scale



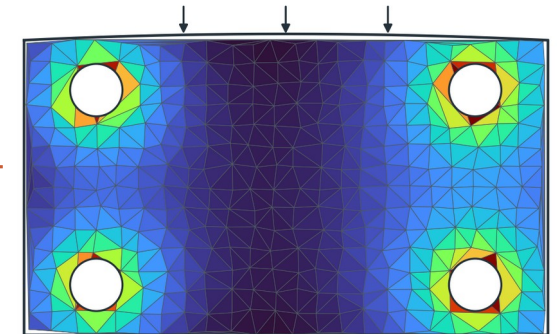
Ion transport
GROMACS · nm
scale



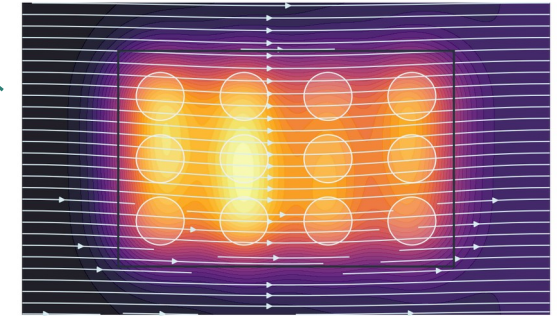
BATTERY PACK



Structural response
CalculiX · mm-m
scale



Flow and heat
OpenFOAM · cm-m
scale



energy density · safety · lifetime

*Four models exchange properties, heat sources
and mechanical loads*

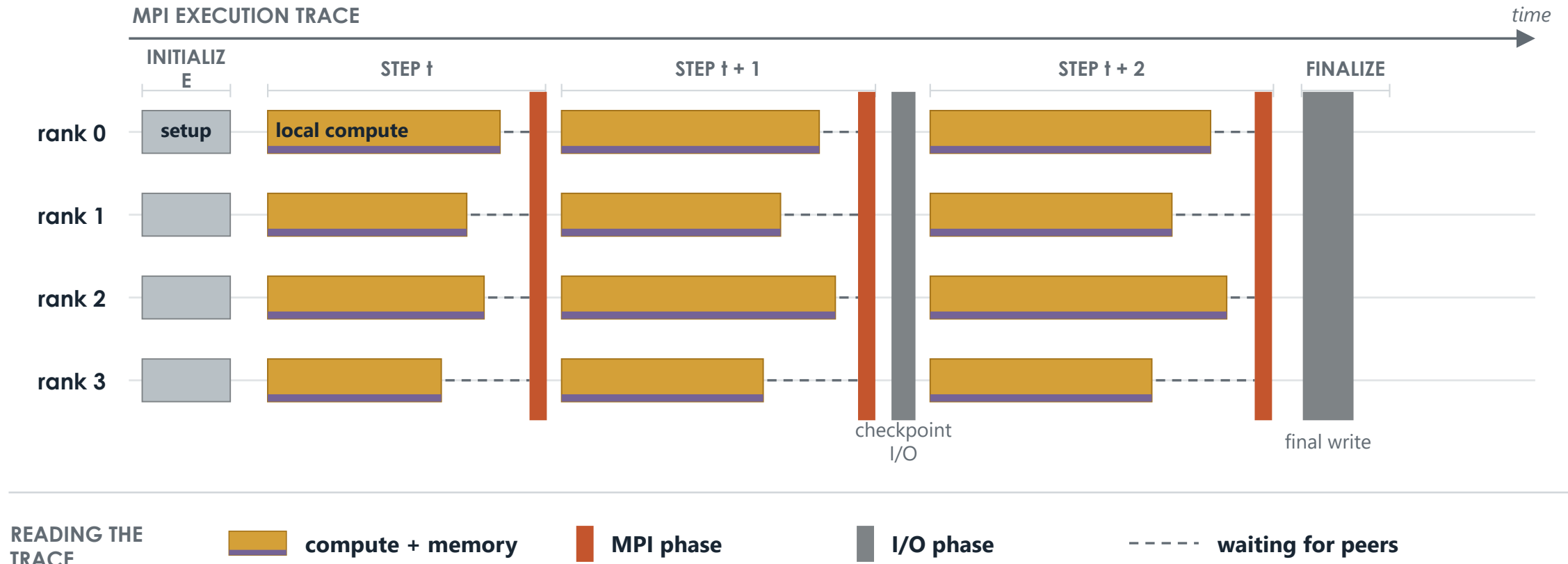
Running Scientific Software on the Cluster

Modules, containers, Slurm and choosing the right resources for the job



Inside a Running Simulation

Across MPI ranks, local kernels run in parallel; synchronization punctuates the timeline



The slowest rank sets the pace at every synchronization.

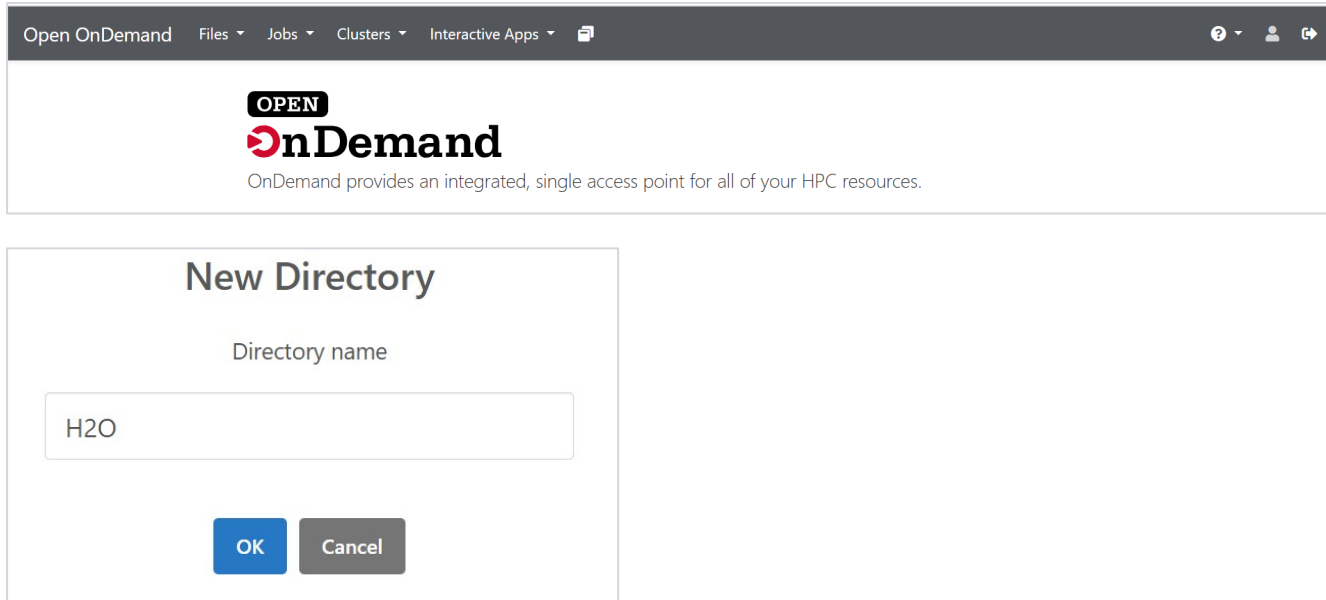
Finding and Loading Software

```
bash – login node

$ module avail    # everything installed on the cluster
$ module spider qe # which versions exist?
$ module load gcc/12.2 openmpi/4.1
$ module load qe/7.3.1
$ module list     # what is active right now
$ which gmx_mpi   # confirm the binary you will run
$ module purge    # start from a clean environment
```

- Modules let many compiler, MPI and application versions coexist.
- Load the same stack you benchmarked with — and record it, because a different MPI build can change your results and your timings.

Getting onto the Cluster



The screenshot shows the Open OnDemand web interface. At the top, there is a navigation bar with links: Open OnDemand, Files, Jobs, Clusters, and Interactive Apps. Below the navigation bar is the Open OnDemand logo and a tagline: "OnDemand provides an integrated, single access point for all of your HPC resources." In the center, there is a "New Directory" dialog box. It has a title "New Directory" and a label "Directory name". Below the label is a text input field containing "H2O". At the bottom of the dialog box are two buttons: "OK" and "Cancel".

Namal HPC — Open OnDemand, and a new working directory created for the water calculation

How the run was set up

- **Browser access** — Open OnDemand — no SSH client needed.
- **Files and Jobs** — upload, edit and submit from one page.
- **Workspace** — /mnt/lustre/mushafia on the parallel file system.
- **One folder per system** — H2O, NH3 and H2 kept separate.
- **Inside each folder** — the input file, the pseudopotentials, the job script.

The Input File: H2O.in

/mnt/lustre/hpcscool26/mushafia/qe test/h2o.in

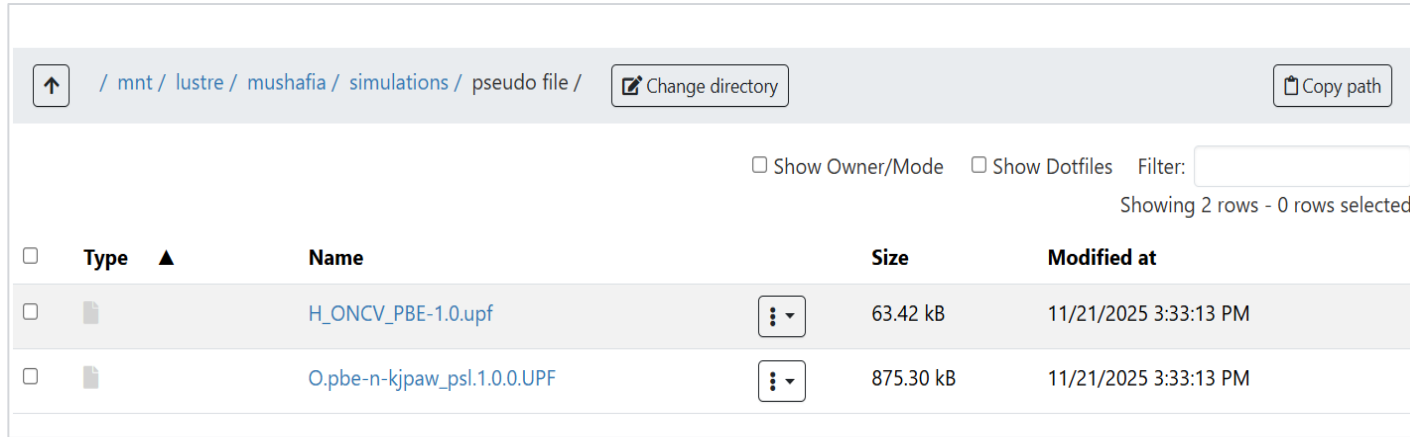
```
1 &control
2   calculation = 'relax'
3   prefix = 'h2o'
4   pseudo_dir = '/mnt/lustre/hpcscool26/mushafia/pseudofile'
5   etot_conv_thr = 1e-5
6   forc_conv_thr = 1e-4
7 /
8 &system
9  ibrav=1, celldm(1) =10,
10  nat=3, ntyp=2,
11  ecutwfc=25
12 /
13 &electrons
14   conv_thr=1e-8
15 /
16 &ions
17   ion_dynamics='bfgs'
18 /
19 /
20 ATOMIC_SPECIES
21  H  1.00784  H_ONCV_PBE-1.0.upf
22  O  15.999  O.pbe-n-kjpaw_psl.1.0.0.UPF
23 ATOMIC_POSITIONS (angstrom)
24  O 0.00 0.00 0.00
25  H -1.00 0.01 0.02
26  H 1.00 0.01 0.03
27 K_POINTS (automatic)
28  1 1 1 0 0 0
```

What each block does

- **&control** — calculation, prefix, pseudo_dir and convergence thresholds.
- **&system** — cell, nat = 3, ntyp = 2, ecutwfc = 25 Ry.
- **&electrons** — conv_thr = 1e-8 for the SCF cycle.
- **&ions** — BFGS relaxation of the geometry.
- **Species and positions** — the atoms, their UPF names and the k-point grid.

/mnt/lustre/mushafia/simulations/examples/H2O.in — relax run, 3 atoms, 25 Ry cutoff

Pseudopotentials: One File per Element



The screenshot shows a file manager interface with a breadcrumb path: /mnt / lustre / mushafia / simulations / pseudo file /. It includes buttons for 'Change directory' and 'Copy path'. Below the path, there are checkboxes for 'Show Owner/Mode' and 'Show Dotfiles', and a 'Filter:' input field. A status bar indicates 'Showing 2 rows - 0 rows selected'. The main table lists two files:

☐	Type ▲	Name		Size	Modified at
☐	📄	H_ONCV_PBE-1.0.upf	⋮	63.42 kB	11/21/2025 3:33:13 PM
☐	📄	O.pbe-n-kjpaw_psl.1.0.0.UPF	⋮	875.30 kB	11/21/2025 3:33:13 PM

The pseudo directory holding the hydrogen and oxygen UPF files used by this run

Where first runs break

- **One UPF per element** — here H and O, 63 kB and 875 kB.
- **Keep them together** — a pseudo directory beside the input file.
- **Point at it** — pseudo_dir in the &control block.
- **Names must match exactly** — the string in ATOMIC_SPECIES is the file name.
- **Record the set** — a different pseudopotential gives a different number.

Choosing Resources for a Job

How many cores?



Scaling test first

Run 1, 2, 4, 8 ranks and watch where the speed-up flattens.

How much memory?



Estimate from the problem

Atoms, mesh cells, basis size, matrices and number of replicas.

Do I need a GPU?



Only if the build uses it

Check that your dominant kernels are actually offloaded.

How many nodes?



Only when it pays

Multi-node helps when compute per rank outweighs communication.

Match the request to the workload, not to what the cluster happens to have

Submitting the Quantum ESPRESSO Job

job.slurm – the script used for the water run

/mnt/lustre/mushafia/simulations/examples/job.slurm

```
1  #!/bin/bash
2  #SBATCH --job-name=test
3  #SBATCH --nodes=1
4  #SBATCH --partition=gpu
5  #SBATCH --time=00:05:00
6  #SBATCH --ntasks-per-node=1
7  #SBATCH --cpus-per-task=1
8  #SBATCH --mem=16G
9  #SBATCH --gres=gpu:1
10 #SBATCH --output=gpu_test_%j.out
11 #SBATCH --error=gpu_test_%j.err
12 module load qe/7.3.1
13 module load openmpi/4.1
14
15
16 mpirun -np 1 pw.x -inp H2O.in > H2O.out
17
```

```
-----
mushafia@master-node's password:
[mushafia@master-node examples]$ sbatch job.slurm
Submitted batch job 2421
```


Monitoring & Performance

A job that finishes is not the same as a job that ran well — watch it,
measure it, then scale it



Watching a Job While It Runs

```
bash – login node

$ squeue -u $USER # queued or running?
$ squeue -j 48213 --start # estimated start time
$ sstat -j 48213 --format=JobID,AveCPU,MaxRSS # live CPU & memory
$ srun --jobid=48213 --pty htop # per-core load on the node
$ srun --jobid=48213 --pty nvidia-smi # GPU utilisation & memory
$ tail -f logs/48213.out # the application's own progress
$ scancel 48213 # stop a job that is misbehaving
$ sacct -j 48213 --format=JobID,Elapsed,TotalCPU,MaxRSS,State
```

- Two questions while it runs: are the cores you reserved actually busy, and is the GPU actually being used?
- Idle reserved hardware is the most common waste on a shared cluster.

Cluster Monitoring with Grafana



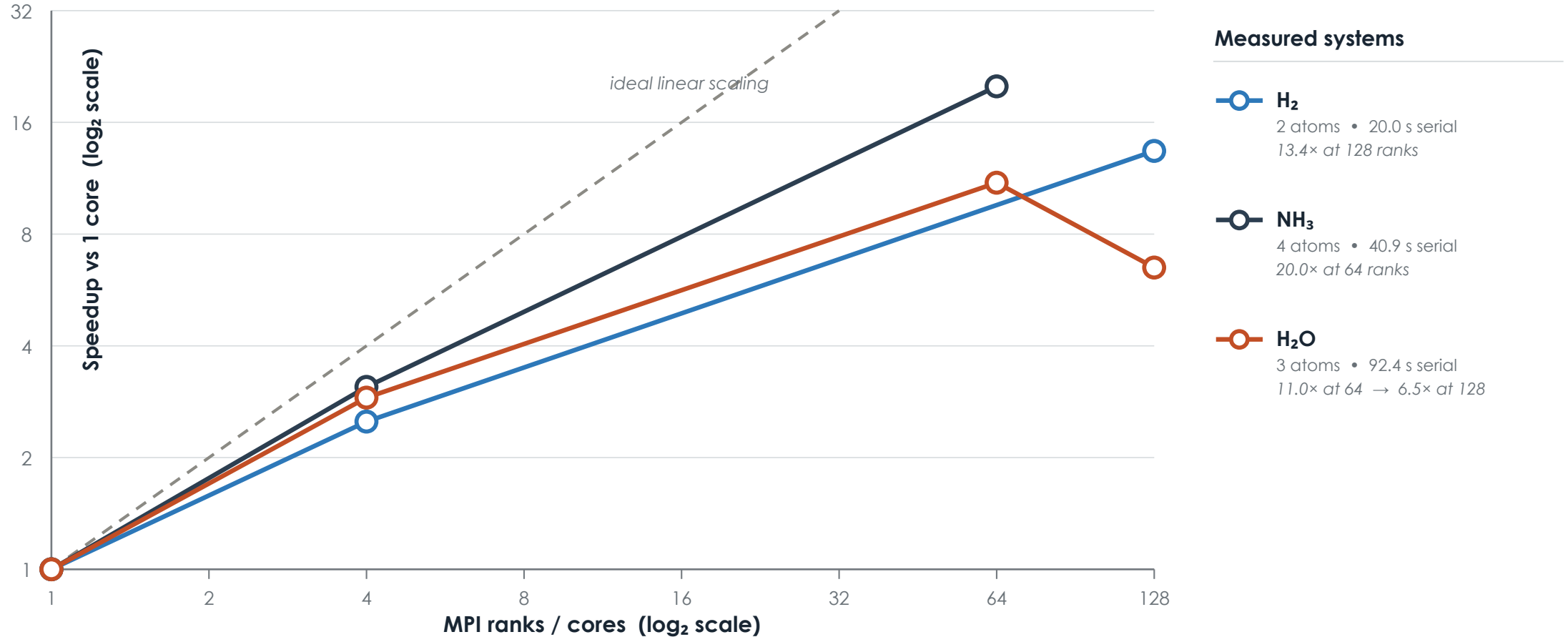
Namal HPC — NVIDIA DCGM dashboard: GPU temperature and power across node01–node04

What to look for

- **Node CPU load** — Are all the cores you reserved actually busy?
- **Memory usage** — Comfortable headroom, or about to be killed?
- **GPU utilisation** — Is the accelerator working, or just allocated?
- **Network traffic** — High and constant usually means communication-bound.
- **Node health** — Temperature, uptime and failures across the cluster.

Measured Scaling on Namal HPC

Strong scaling of Quantum ESPRESSO test systems • markers are measured runs; lines guide the eye



Measured ranks — H₂: 1, 4, 128 • NH₃: 1, 4, 64 • H₂O: 1, 4, 64, 128. Unequal point counts reflect the runs performed.

Where This Matters for Pakistan

Agriculture & water



Soil and irrigation modelling

Crop systems, molecular plant science, food processing.

Energy & materials



Battery and solar materials

Turbines, fans, thermal and fluid systems.

Healthcare



Protein dynamics and drug binding

Bioinformatics pipelines and medical data analytics.

Climate & engineering



Weather, water and flooding

Manufacturing, transport and structural design.

The same cluster and the same open software, pointed at national problems

Why Open-Source Software Matters Here

- No per-seat licence cost — the budget goes to hardware and people instead
- Source access allows inspection, modification and local optimisation
- The same stack deploys on a lab workstation and on a national cluster
- Reading real solver code teaches numerical methods, compilers and parallelism
- Open code and open inputs let other institutions reproduce the result
- A route from being software users to becoming software developers

Capability keep — not a license have to renew

“

Open scientific software plus local HPC turns national problems — crops, water, batteries, turbines, medicine, weather — into questions we can compute for ourselves.

— Open software + local HPC + skilled people = indigenous computational capability

Key Takeaways

- Scientific software is validated physics plus numerical methods plus an optimised parallel implementation.
- GROMACS, Quantum ESPRESSO and OpenFOAM represent three families: particles, electrons and meshes.
- MPI, OpenMP, GPUs, numerical libraries and Slurm are the shared machinery underneath all of them.
- Choose resources from measured scaling — and check seff, logs and Grafana after every run.
- Modules, containers and a recorded environment are what make the result reproducible.

Next: we do all of this live on the Namal cluster.

Documentation & Further Reading

- **GROMACS** — manual.gromacs.org
- **Quantum ESPRESSO** — quantum-espresso.org/documentation
- **OpenFOAM** — openfoam.com/documentation
- **LAMMPS** — docs.lammps.org
- **Slurm workload manager** — slurm.schedmd.com/documentation.html
- **Apptainer containers** — apptainer.org/docs
- **Grafana dashboards** — grafana.com/docs



Thank You

Questions

Mushafia Sadia Aman | Centre for AI and Big Data, Namal University
